

Types of Inheritance in C++

The 5 important types are:

1. Single
2. Multiple
3. Multilevel
4. Hierarchical
5. Hybrid

The easiest way to remember them is by looking at the **shape of the inheritance tree**.

1. Single Inheritance

One base → one derived class

```
Animal
  ↓
Dog
class Animal {
public:
    void eat() {
        cout << "Eating";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking";
    }
};
```

Here:

Animal → Dog

Dog inherits from **Animal**.

Interview definition

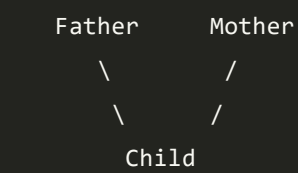
Single inheritance is when one derived class inherits from exactly one base class.

Example

```
Dog d;
d.eat();    // inherited
d.bark();   // own method
```

2. Multiple Inheritance

Multiple base classes → one derived class



```
class Father {
public:
    void skills() {
        cout << "Father skills";
    }
};

class Mother {
public:
    void hobbies() {
        cout << "Mother hobbies";
    }
};

class Child : public Father, public Mother {
};
```

Now `Child` inherits from **both** `Father` and `Mother`.

```
Child c;

c.skills();
c.hobbies();
```

Interview definition

Multiple inheritance is when one derived class inherits from more than one base class.

Important interview issue: Ambiguity

Suppose both parents have the same function:

```
class Father {
public:
    void show() {
        cout << "Father";
    }
};

class Mother {
public:
    void show() {
        cout << "Mother";
    }
};

class Child : public Father, public Mother {
};
```

Then:

```
Child c;
c.show();    // ERROR: ambiguous
```

Because C++ doesn't know which `show()` you mean.

Resolve it using scope resolution:

```
c.Father::show();
c.Mother::show();
```

★ Interview tip

Multiple inheritance is one reason interfaces/abstract classes are commonly discussed in C++.

C++ allows:

```
class C : public A, public B
```

while Java, for example, doesn't allow multiple inheritance of classes.

3. Multilevel Inheritance

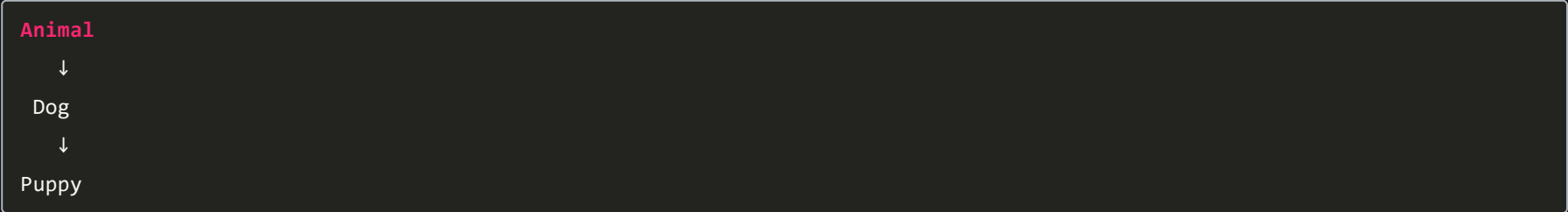
Inheritance happens in a chain.

```
Grandparent
  ↓
  Parent
    ↓
    Child
class Animal {
public:
    void eat() {}
};

class Dog : public Animal {
public:
    void bark() {}
};

class Puppy : public Dog {
public:
    void play() {}
};
```

So:



Puppy gets members from both **Dog** and **Animal**.

```
Puppy p;

p.eat(); // Animal
p.bark(); // Dog
p.play(); // Puppy
```

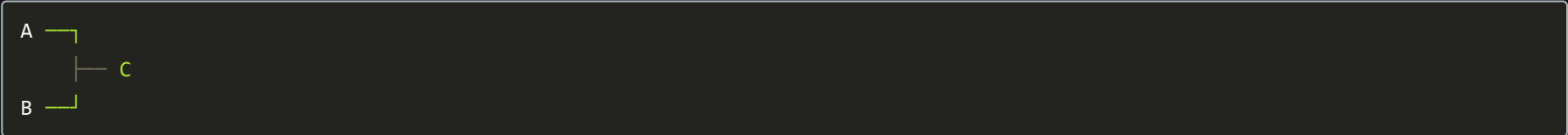
Interview definition

Multilevel inheritance is when a derived class itself becomes the base class for another class.

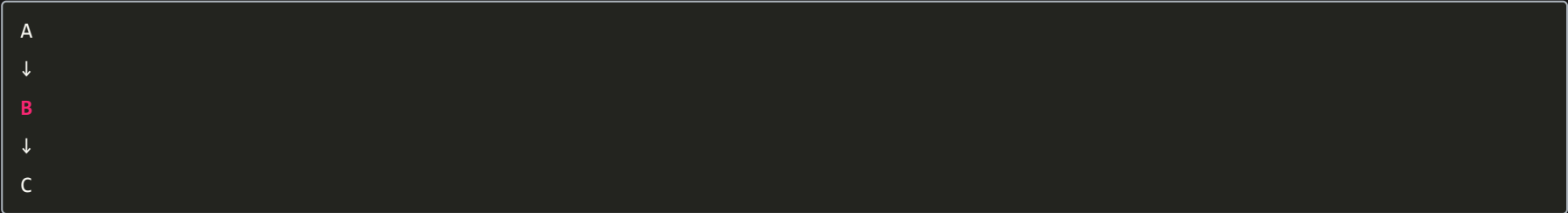
★ Common confusion

Multilevel ≠ multiple

Multiple:



Multilevel:



4. Hierarchical Inheritance

One base → multiple derived classes

```

    Animal
    /    \
  Dog    Cat

class Animal {
public:
    void eat() {}
};

class Dog : public Animal {
public:
    void bark() {}
};

class Cat : public Animal {
public:
    void meow() {}
};
```

Both `Dog` and `Cat` inherit from `Animal`.

```
Dog d;
d.eat();

Cat c;
c.eat();
```

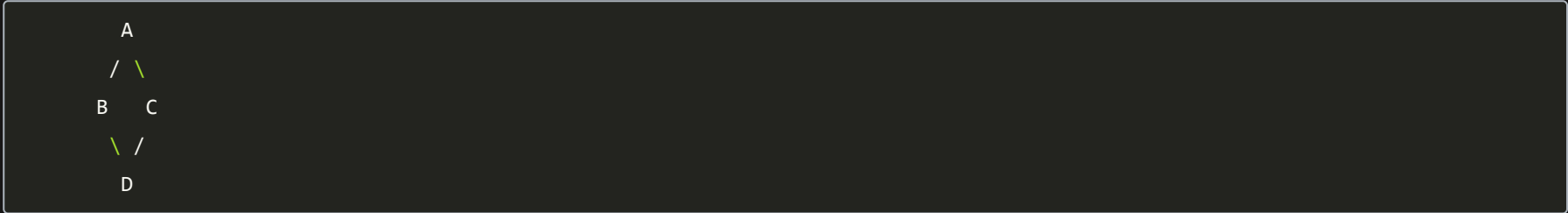
Interview definition

Hierarchical inheritance is when multiple derived classes inherit from the same base class.

5. Hybrid Inheritance

Combination of two or more inheritance types.

For example:



This combines:

- hierarchical inheritance: `A → B` and `A → C`
- multiple inheritance: `B + C → D`

Therefore it's **hybrid inheritance**.

```
class A {
public:
    int x;
};

class B : public A {
};

class C : public A {
};

class D : public B, public C {
};
```

And now we have a major problem.

```
D obj;

obj.x;    // ERROR: ambiguous
```

Why?

Because `D` contains **two copies of `A`**:



This is the famous **Diamond Problem**.

★ Diamond Problem

This is one of the **most important inheritance interview questions**.

```
graph TD
    A --> B
    A --> C
    B --> D
    C --> D
```

```
class A {
public:
    int x;
};

class B : public A {};
class C : public A {};

class D : public B, public C {};
```

Now:

```
D d;
d.x;
```

Compiler asks:

Which **x**? The one inherited through **B** or through **C**?

Hence ambiguity.

Solution: Virtual Inheritance

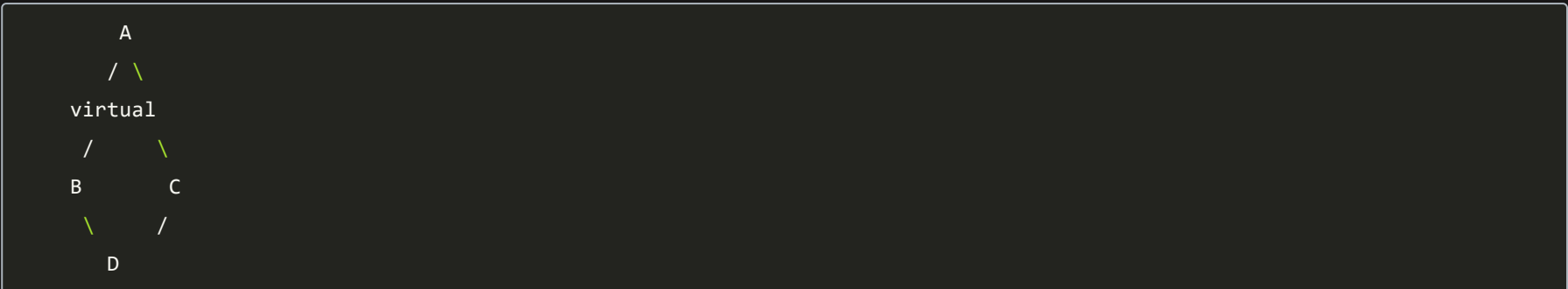
C++ provides **virtual inheritance** to ensure that only **one shared copy** of the common base exists.

```
class A {
public:
    int x;
};

class B : virtual public A {};
class C : virtual public A {};

class D : public B, public C {};
```

Now:



D has only one **A** subobject.

Therefore:

```
D d;
d.x; // OK
```

If asked:

What is the diamond problem in C++ and how do you solve it?

Say:

The diamond problem occurs in multiple inheritance when two intermediate classes inherit from the same base class and a final class inherits from both, resulting in two copies of the base class and ambiguity. C++ solves this using virtual inheritance, which ensures that only one shared instance of the common base class exists.

That's a very good interview answer.

All 5 at a glance

TypeStructureSingleA → BMultipleA + B → CMultilevelA → B → CHierarchicalA → B, A → CHybridCombination of multiple types

Memory trick

Single	: one → one
Multiple	: many → one
Multilevel	: chain
Hierarchical	: one → many
Hybrid	: combination

🌟 Interview Questions You Should Know

Basic

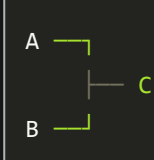
Q1. What are the different types of inheritance in C++?

Answer:

Single, multiple, multilevel, hierarchical, and hybrid inheritance.

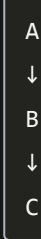
Q2. Difference between multiple and multilevel inheritance?

Multiple:



```
graph BT; A --> C; B --> C
```

Multilevel:



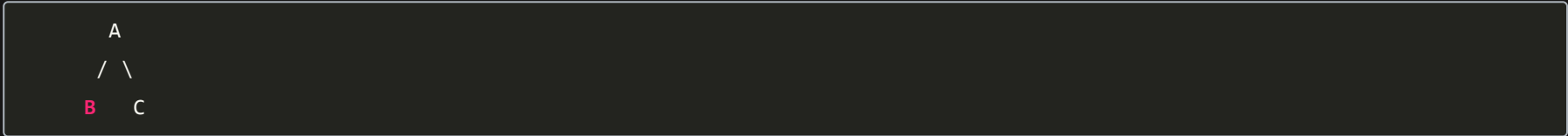
```
graph TD; A --> B; B --> C
```

Multiple = **multiple** parents

Multilevel = **multiple** levels

Q3. What is hierarchical inheritance?

Multiple derived classes inherit from the same base class.



Q4. What is hybrid inheritance?

Hybrid inheritance is a combination of two or more types of inheritance.

★ Intermediate

Q5. What is the diamond problem?

Know this diagram:



Two copies of `A` can reach `D`.

Q6. How does C++ solve the diamond problem?

Virtual inheritance.

```
class B : virtual public A {};  
class C : virtual public A {};
```

Q7. Does C++ support multiple inheritance?

Yes.

```
class C : public A, public B {};
```

Q8. Why can multiple inheritance cause ambiguity?

Because two base classes can contain members with the same name.

```
class A {  
public:  
    void show();  
};  
  
class B {  
public:  
    void show();  
};  
  
class C : public A, public B {};  
C c;  
c.show(); // ambiguous
```

Resolve:

```
c.A::show();  
c.B::show();
```

💧 One important distinction

Don't confuse **type of inheritance** with **access mode**.

This:

```
class Dog : public Animal
```

`public` here specifies the **inheritance access mode**.

It does **not** mean "public inheritance type."

The inheritance **type** is:

```
Animal → Dog
```

which is **single inheritance**.

The `public` determines how the inherited `public`/`protected` members are exposed in `Dog`.

That's a common interview trap.

What you should be able to draw from memory

For interviews, memorize these five diagrams:

1. Single

A

↓

B

2. **Multiple**

A **B**

 \ /

 C

3. **Multilevel**

A

↓

B

↓

C

4. Hierarchical

 A

 / \

B C

5. Hybrid / Diamond

 A

 / \

B C

 \ /

 D

And for **hybrid/diamond**, immediately think:

"Virtual inheritance → one shared base."